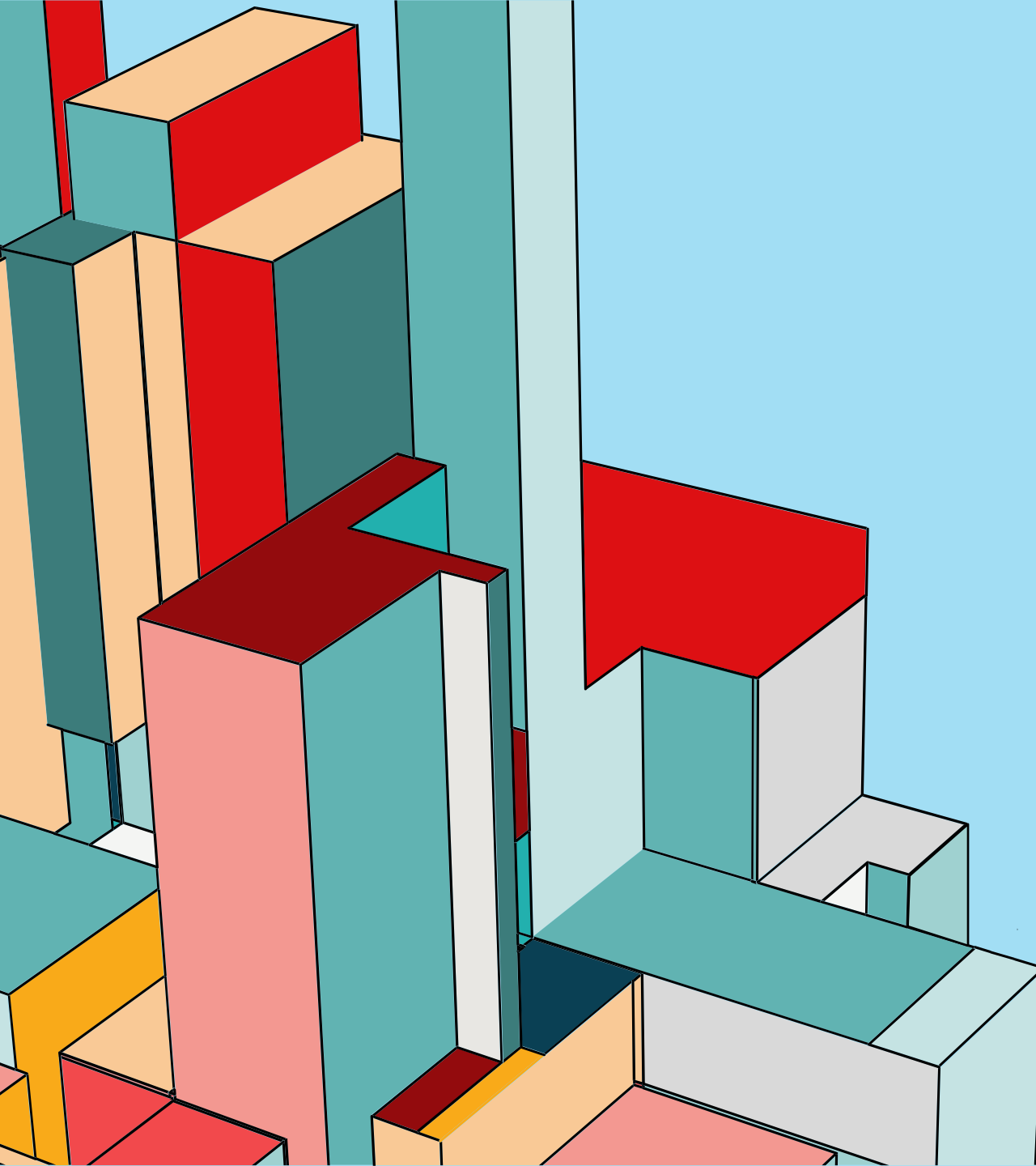


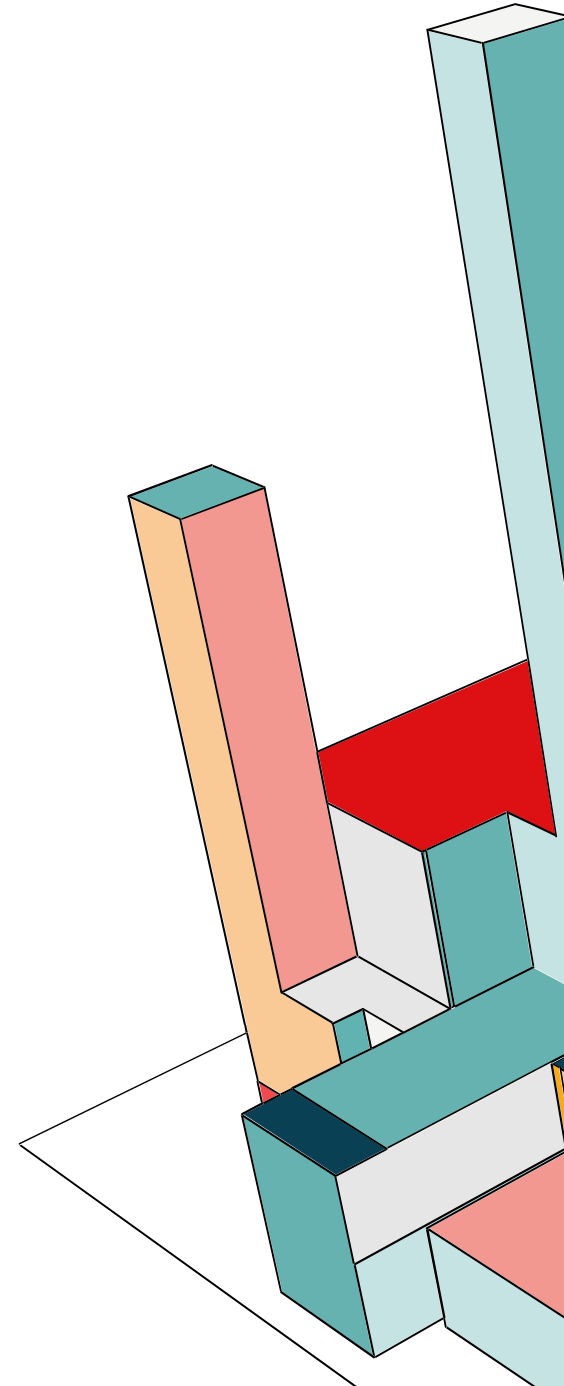


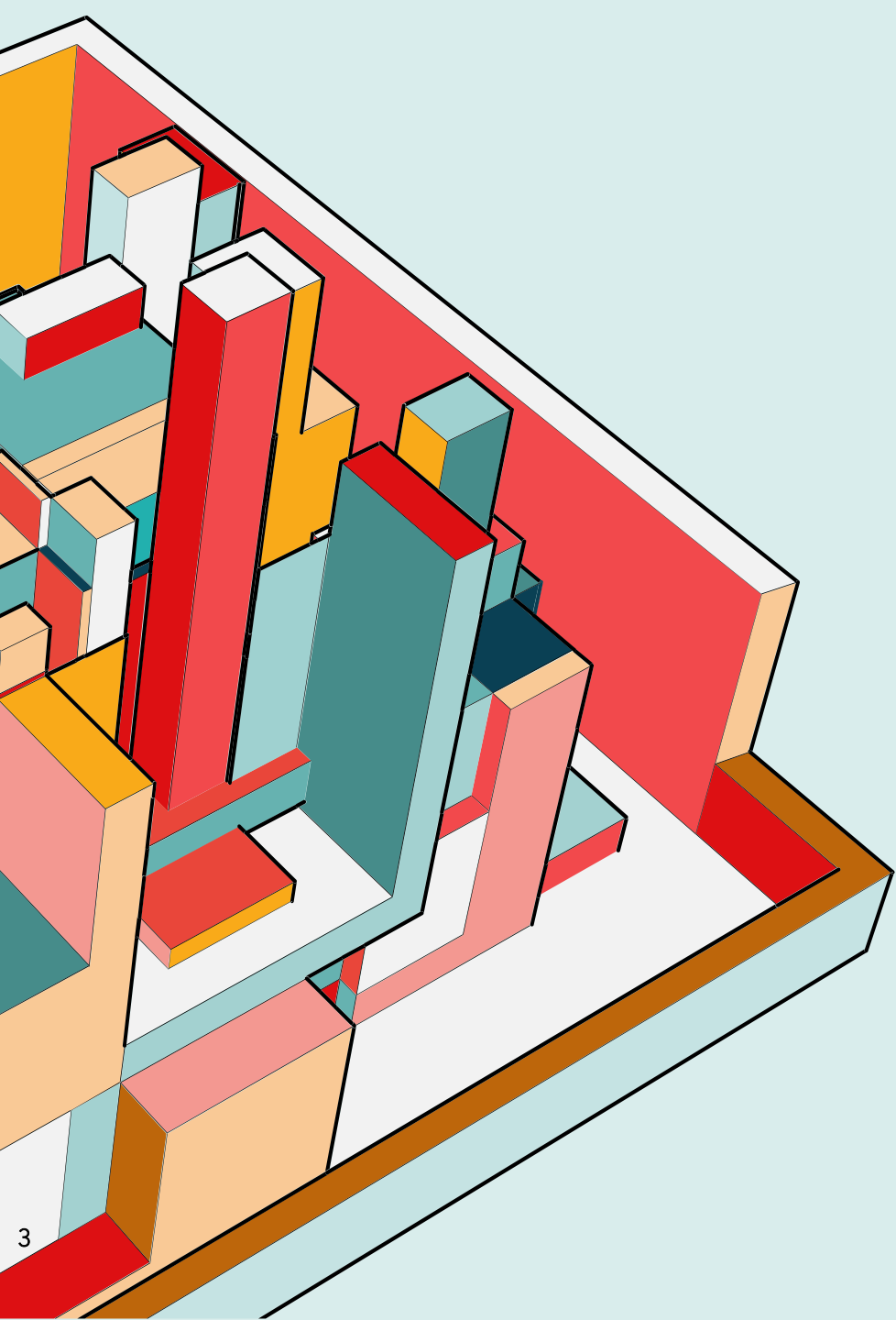
LECTURE 11 **MORE PATH FINDING; HEAPS**

CSIS 2300 Introduction to
Algorithms, Spring 2026
Dr. Dan DiTursi

ANNOUNCEMENTS

- Grade inquiries
- Spring Break
- Advising meetings

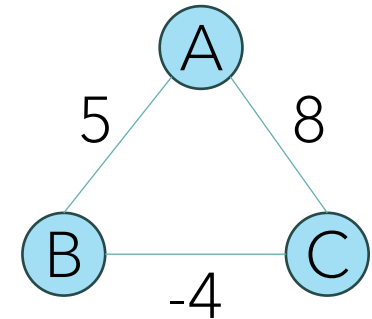




THE BELLMAN-FORD ALGORITHM

REMINDER FROM LAST CLASS

Dijkstra's algorithm does not work if there are negative edge weights in the graph.



Say you found that the shortest path from A to B was 5, and then later that the shortest path from A to C was 8. But if there's an edge from C to B with weight -4, there is now a shorter path to a "finalized" node.

Furthermore, if there is a negative cycle in the graph, this renders the entire concept of "shortest path" useless – you can just go around the cycle as many times as you like.

Thus, in cases where negative edge weights are possible, we need a new algorithm.

REQUIREMENTS OF THIS NEW ALGORITHM

- Detect negative cycles and report that there is no solution when they are present.
- If there is no negative cycle, report the correct shortest path and its distance.
- Ideally run in the same speed as Dijkstra's algorithm; at the very least, it shouldn't be a LOT slower.

DIJKSTRA'S DOWNFALL...

... AND THE SEEDS OF A NEW ALGORITHM

In Dijkstra's algorithm, the distance values that we track for each node are always either correct or overestimates.

Recall that, for any vertex v other than the source, it starts at ∞ and only gets updated when we look at one of the neighbors of v .

In a very real sense, Dijkstra's algorithm is just repeated invocations of a very simple method we can call **update**:

```
def update(u,v): # (u,v) is an edge in G
    dist(v) = min(dist(v), dist(u) + w(u,v))
```

This method never makes $\text{dist}(v)$ too small, which means it is, in some sense, "safe" – we can call it as many times as we want without screwing anything up.

DIJKSTRA'S DOWNFALL...

... AND THE SEEDS OF A NEW ALGORITHM

```
def update(u,v): # (u,v) is an edge in G
    dist(v) = min(dist(v), dist(u) + w(u,v))
```

This method sets **dist(v)** correctly when (1) the edge **(u,v)** is the last link in the path from the source node to **v**, and (2) **dist(u)** is set correctly.

Also, it never makes **dist(v)** too small, which means it is, in some sense, "safe" – we can call it as many times as we want without screwing anything up. For Dijkstra, this doesn't matter – we call it exactly enough times, using the optimal sequence of edges... as long as all edges are positive.

But we don't have to use that sequence. Our trick for handling negative edge weights will simply be to perform more updates.

A REALLY BASIC PROPERTY

If there are no negative cycles in the graph, what can we state for certain about the shortest path between two arbitrary vertices a and b ?

In particular, if the graph has n vertices and m edges, what is the maximum number of edges ℓ that the optimal path from a to b can contain?

Answer this question in the Submittity poll now.



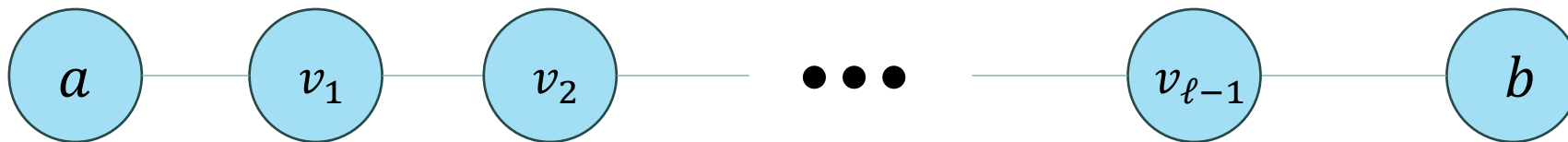
A REALLY BASIC PROPERTY

If there are no negative cycles in the graph, what can we state for certain about the shortest path between two arbitrary vertices a and b ?

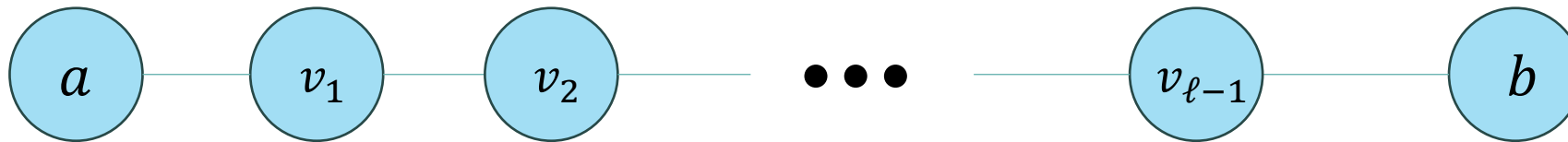
In particular, if the graph has n vertices and m edges, what is the maximum number of edges ℓ that the optimal path from a to b can contain?

Well, it will visit each other node at most one time.

Thus, it will make at most $n - 1$ hops.



GETTING THE ORDER RIGHT



If we were to **update** the distances in exactly this order $(v_1, v_2, \dots, v_{\ell-1}, b)$, then we would get **dist(b)** correct...

... but, of course, if we knew what order to do these updates in, we would already know the optimal path!

How can we ensure that we update v_1 before v_2 before v_3 etc. when we don't even know which vertices they are?

Recall that extra updates won't mess anything up.

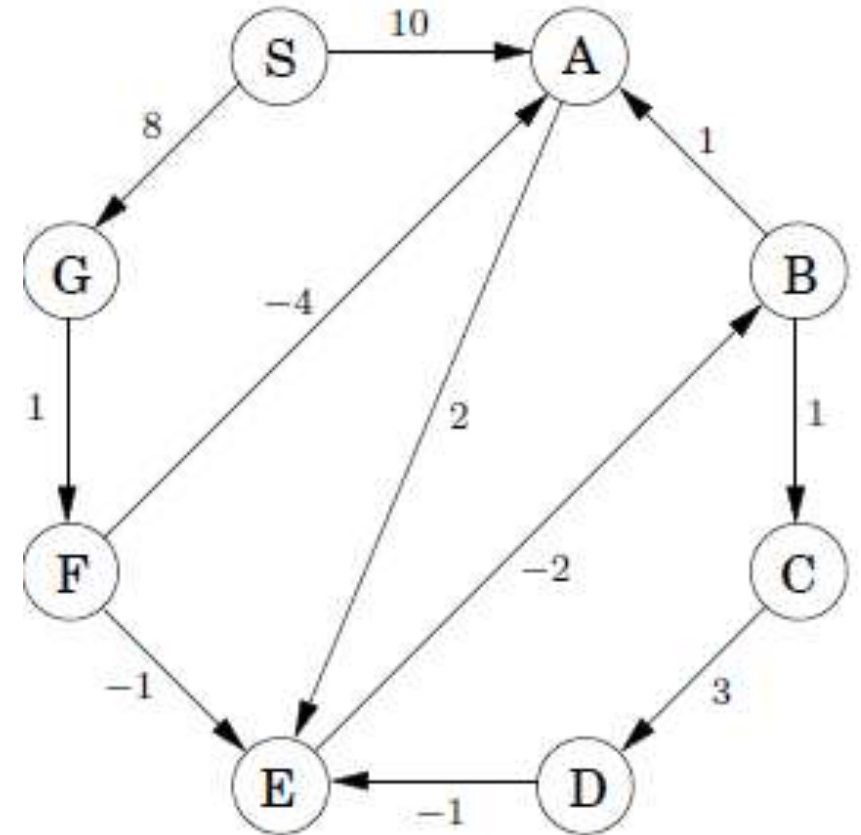
Guess we'll just have to update all of the edges. Repeatedly.

THE BELLMAN-FORD ALGORITHM

```
def BellmanFord(G, v0):  
    Set  $\text{dist}(v) = \infty$ ,  $\text{parent}(v) = \text{null}$  for all  $v$  in  $V$   
    Set  $\text{dist}(v0) = 0$   
    do  $|V|-1$  times:  
        for every edge  $(u,v)$  in  $E$ : # also  $(v,u)$  if undir.  
             $d = \text{dist}(u) + \text{weight}(u,v)$   
            if  $d < \text{dist}(v)$ :  
                 $\text{dist}(v) = d$ ;  $\text{parent}(v) = u$ 
```

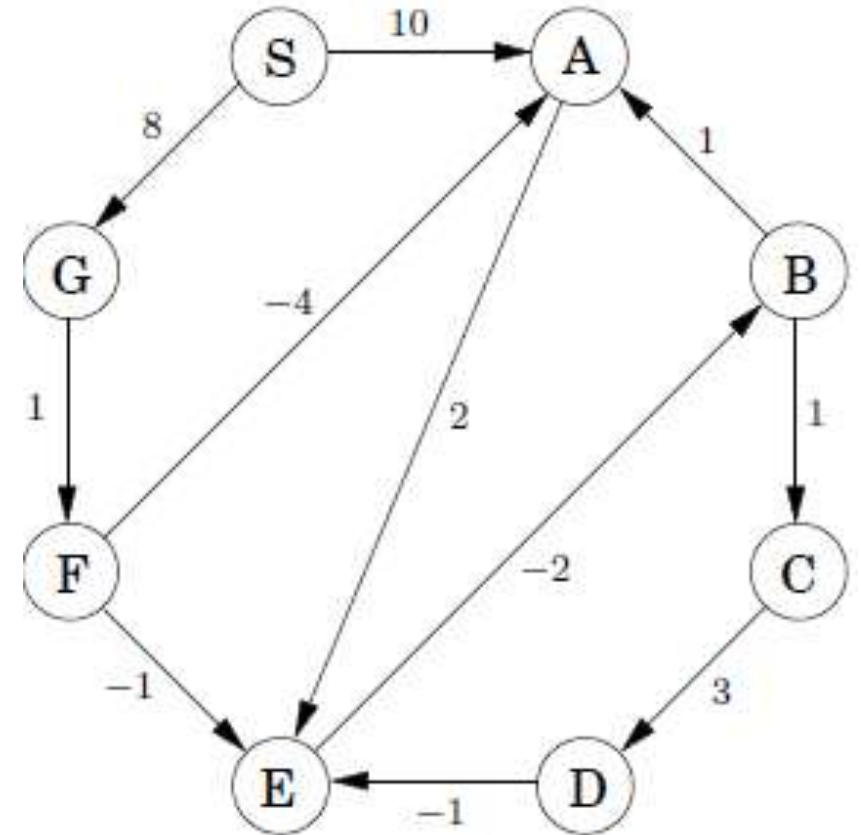
THE BELLMAN-FORD ALGORITHM

	Round							
Node	0	1	2	3	4	5	6	7
S	0,--							
A	∞							
B	∞							
C	∞							
D	∞							
E	∞							
F	∞							
G	∞							



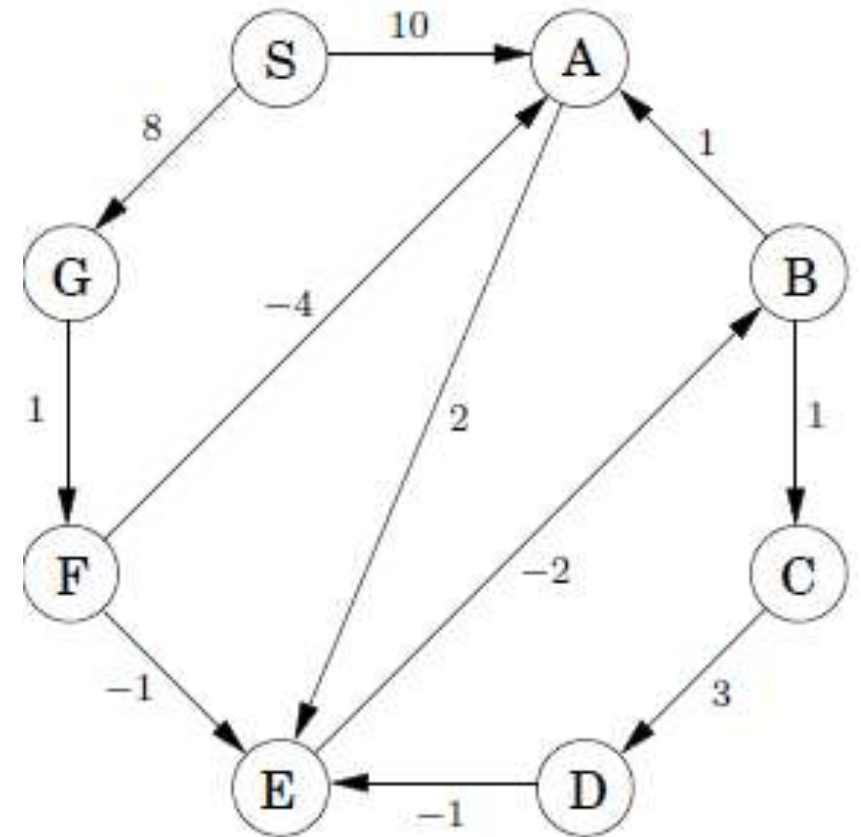
THE BELLMAN-FORD ALGORITHM

	Round							
Node	0	1	2	3	4	5	6	7
S	0,--	0,--						
A	∞	10,S						
B	∞	∞						
C	∞	∞						
D	∞	∞						
E	∞	∞						
F	∞	∞						
G	∞	8,S						



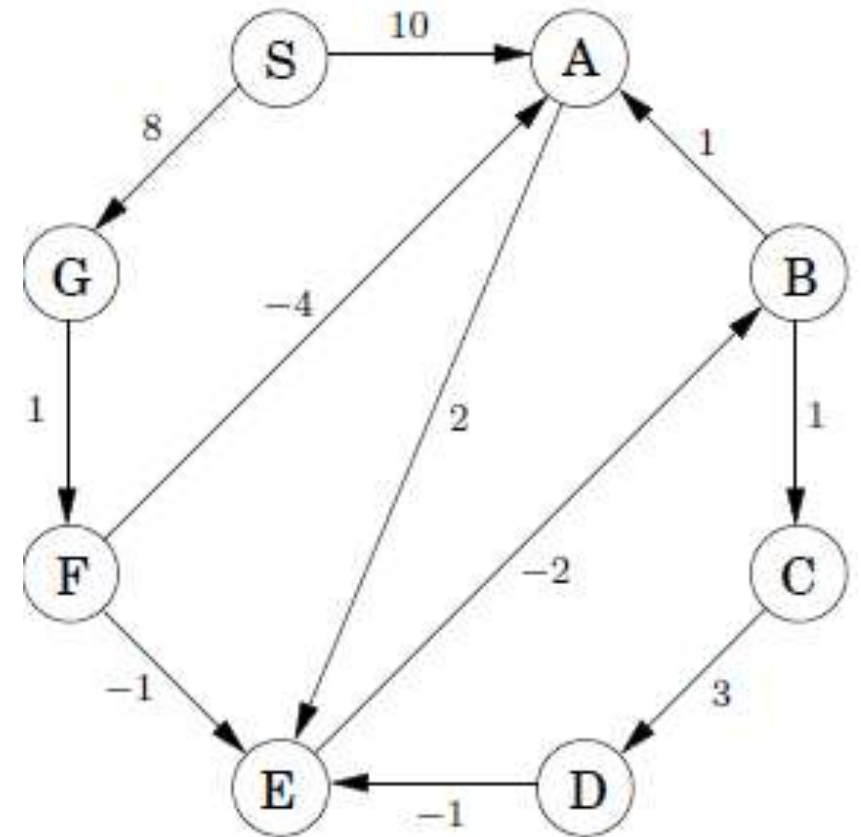
THE BELLMAN-FORD ALGORITHM

	Round							
Node	0	1	2	3	4	5	6	7
S	0,--	0,--	0,--					
A	∞	10,S	10,S					
B	∞	∞	∞					
C	∞	∞	∞					
D	∞	∞	∞					
E	∞	∞	12,A					
F	∞	∞	9,G					
G	∞	8,S	8,S					



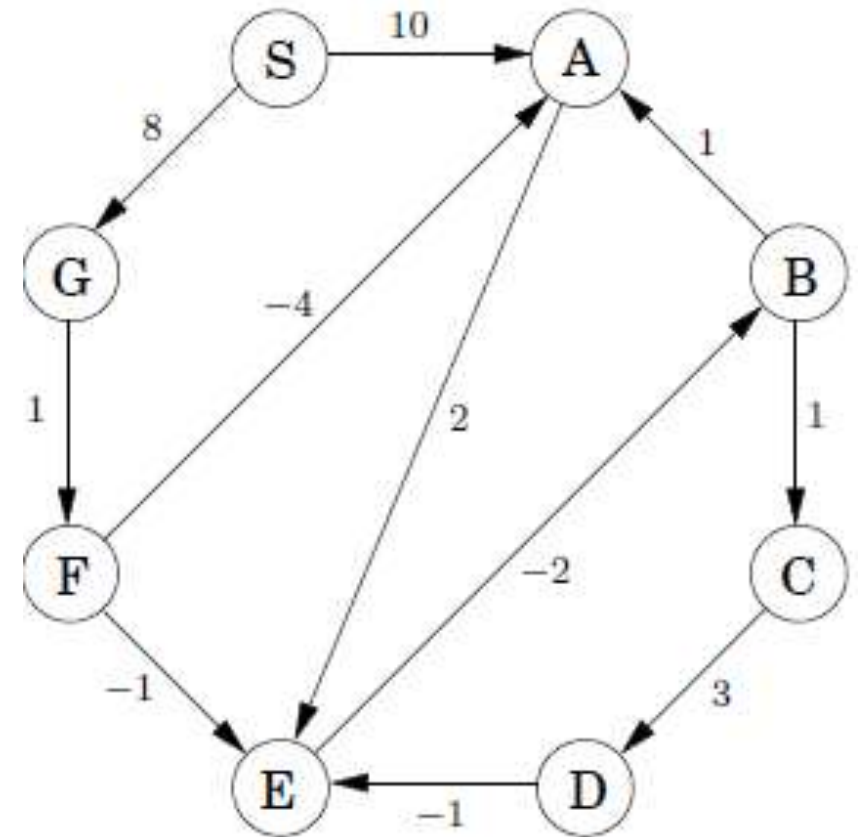
THE BELLMAN-FORD ALGORITHM

	Round							
Node	0	1	2	3	4	5	6	7
S	0,--	0,--	0,--	0,--				
A	∞	10,S	10,S	5,F				
B	∞	∞	∞	10,E				
C	∞	∞	∞	∞				
D	∞	∞	∞	∞				
E	∞	∞	12,A	8,F				
F	∞	∞	9,G	9,G				
G	∞	8,S	8,S	8,S				



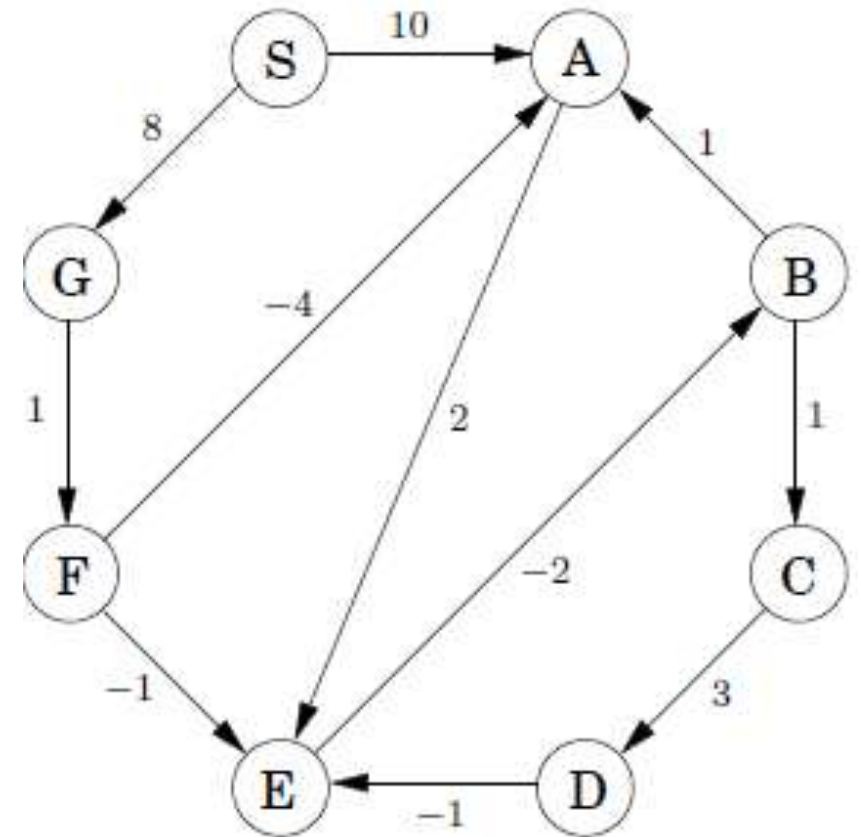
THE BELLMAN-FORD ALGORITHM

	Round							
Node	0	1	2	3	4	5	6	7
S	0,--	0,--	0,--	0,--	0,--			
A	∞	10,S	10,S	5,F	5,F			
B	∞	∞	∞	10,E	6,E			
C	∞	∞	∞	∞	11,B			
D	∞	∞	∞	∞	∞			
E	∞	∞	12,A	8,F	7,A			
F	∞	∞	9,G	9,G	9,G			
G	∞	8,S	8,S	8,S	8,S			



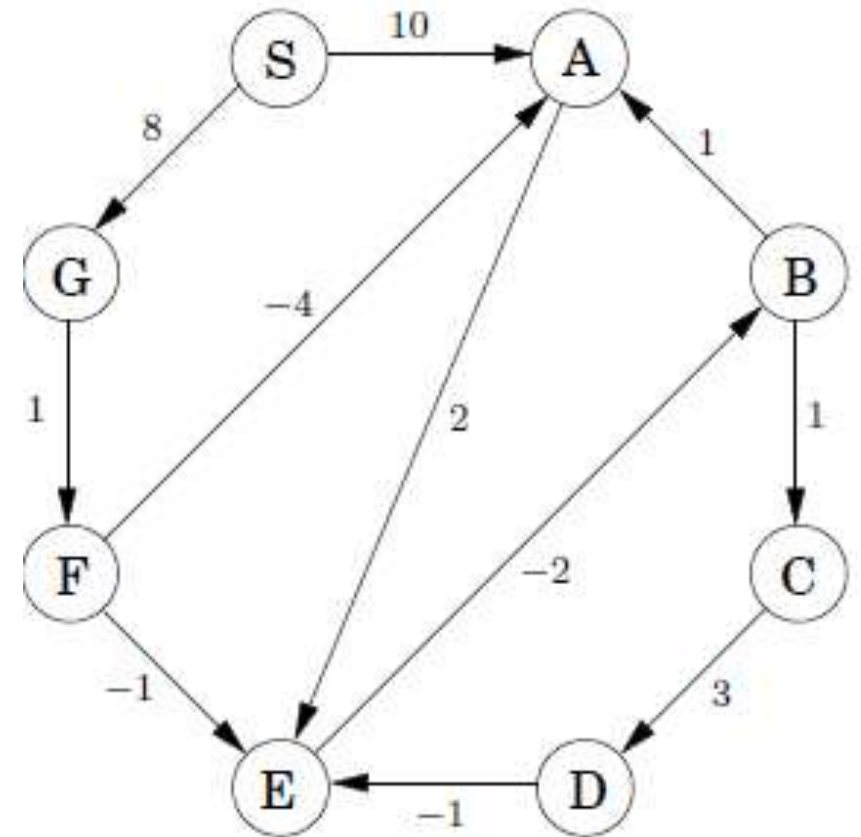
THE BELLMAN-FORD ALGORITHM

	Round							
Node	0	1	2	3	4	5	6	7
S	0,--	0,--	0,--	0,--	0,--	0,--		
A	∞	10,S	10,S	5,F	5,F	5,F		
B	∞	∞	∞	10,E	6,E	5,E		
C	∞	∞	∞	∞	11,B	7,B		
D	∞	∞	∞	∞	∞	14,C		
E	∞	∞	12,A	8,F	7,A	7,A		
F	∞	∞	9,G	9,G	9,G	9,G		
G	∞	8,S	8,S	8,S	8,S	8,S		



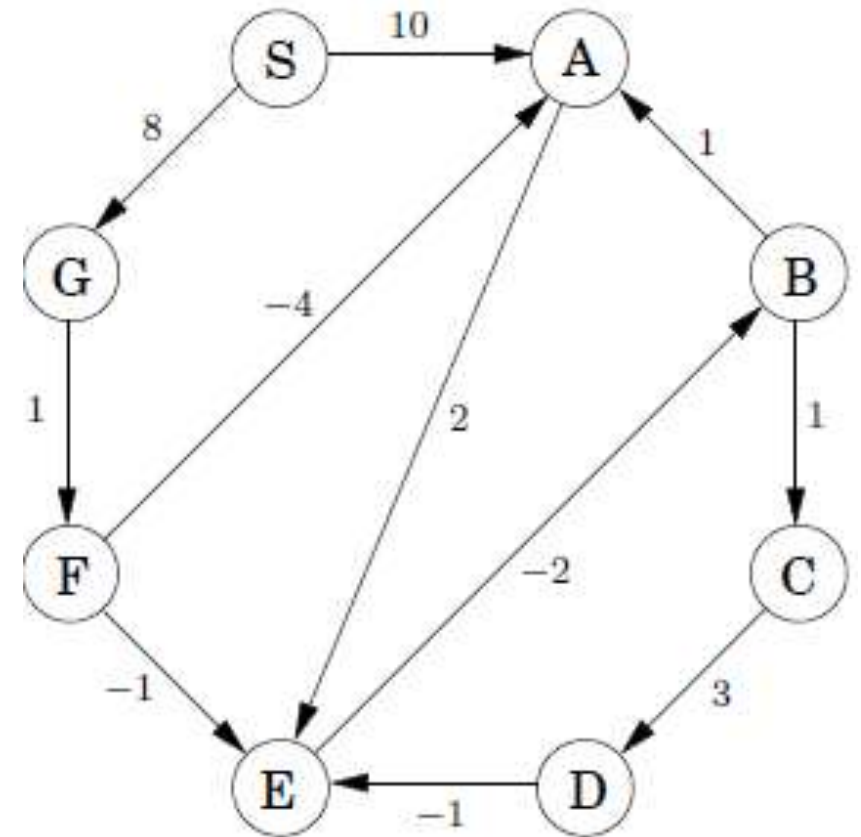
THE BELLMAN-FORD ALGORITHM

	Round							
Node	0	1	2	3	4	5	6	7
S	0,--	0,--	0,--	0,--	0,--	0,--	0,--	
A	∞	10,S	10,S	5,F	5,F	5,F	5,F	
B	∞	∞	∞	10,E	6,E	5,E	5,E	
C	∞	∞	∞	∞	11,B	7,B	6,B	
D	∞	∞	∞	∞	∞	14,C	10,C	
E	∞	∞	12,A	8,F	7,A	7,A	7,A	
F	∞	∞	9,G	9,G	9,G	9,G	9,G	
G	∞	8,S	8,S	8,S	8,S	8,S	8,S	



THE BELLMAN-FORD ALGORITHM

	Round							
Node	0	1	2	3	4	5	6	7
S	0,--	0,--	0,--	0,--	0,--	0,--	0,--	0,--
A	∞	10,S	10,S	5,F	5,F	5,F	5,F	5,F
B	∞	∞	∞	10,E	6,E	5,E	5,E	5,E
C	∞	∞	∞	∞	11,B	7,B	6,B	6,B
D	∞	∞	∞	∞	∞	14,C	10,C	9,C
E	∞	∞	12,A	8,F	7,A	7,A	7,A	7,A
F	∞	∞	9,G	9,G	9,G	9,G	9,G	9,G
G	∞	8,S	8,S	8,S	8,S	8,S	8,S	8,S



Note that another round produces no changes. In fact, we can improve efficiency by adding a check each round to see if any changes were made (like in bubble sort).

OBSERVATIONS ABOUT BELLMAN-FORD

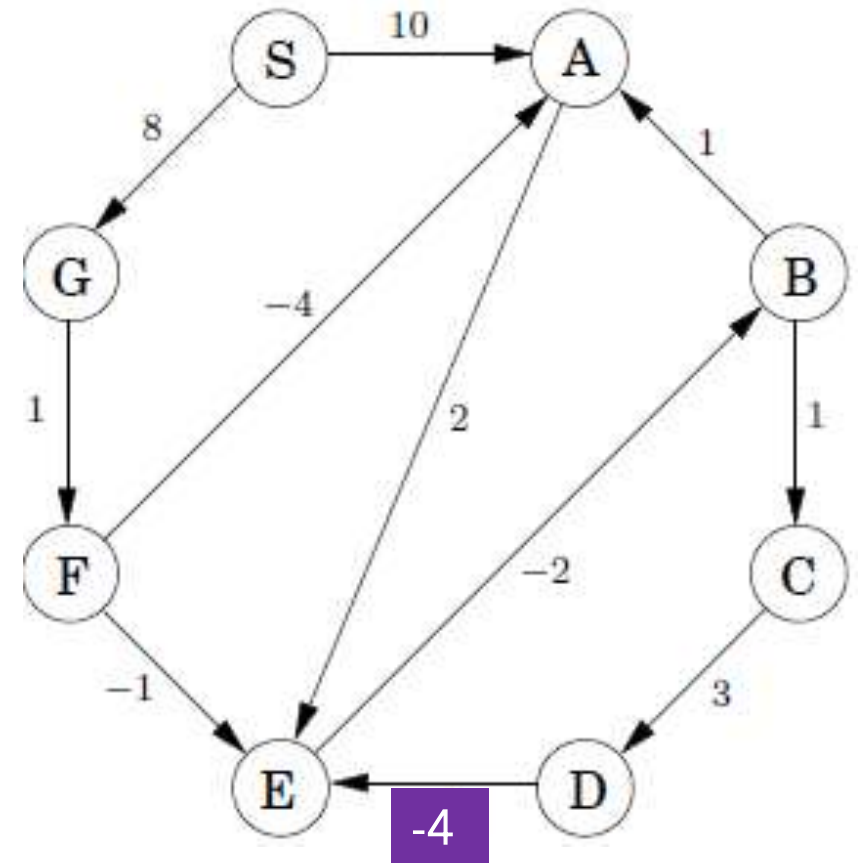
Note how the impact of negative edges propagates along paths in the graph. Also note that we had changes every round precisely because the optimal path to one of the nodes had $|V| - 1$ edges.

The runtime is clearly $\Theta(|V||E|)$ – we look at every edge for $|V| - 1$ iterations.

Manually executing the algorithm makes it less obvious that we need to look at every edge in every round. It might seem like we can cleverly track where the updates have recently happened and work from there. This is true, but in the worst case, it'll still be a large fraction of every node in the graph in most iterations.

NEGATIVE CYCLES

	Round							
Node	0	1	2	3	4	5	6	7
S	0,--	0,--	0,--	0,--	0,--	0,--	0,--	0,--
A	∞	10,S	10,S	5,F	5,F	5,F	5,F	5,F
B	∞	∞	∞	10,E	6,E	5,E	5,E	5,E
C	∞	∞	∞	∞	11,B	7,B	6,B	6,B
D	∞	∞	∞	∞	∞	14,C	10,C	9,C
E	∞	∞	12,A	8,F	7,A	7,A	7,A	7,A
F	∞	∞	9,G	9,G	9,G	9,G	9,G	9,G
G	∞	8,S	8,S	8,S	8,S	8,S	8,S	8,S



What happens if we run another round now?

DETECTING NEGATIVE CYCLES

The baseline pseudocode we looked at previously has an easy update to detect negative cycles: Run one more round.

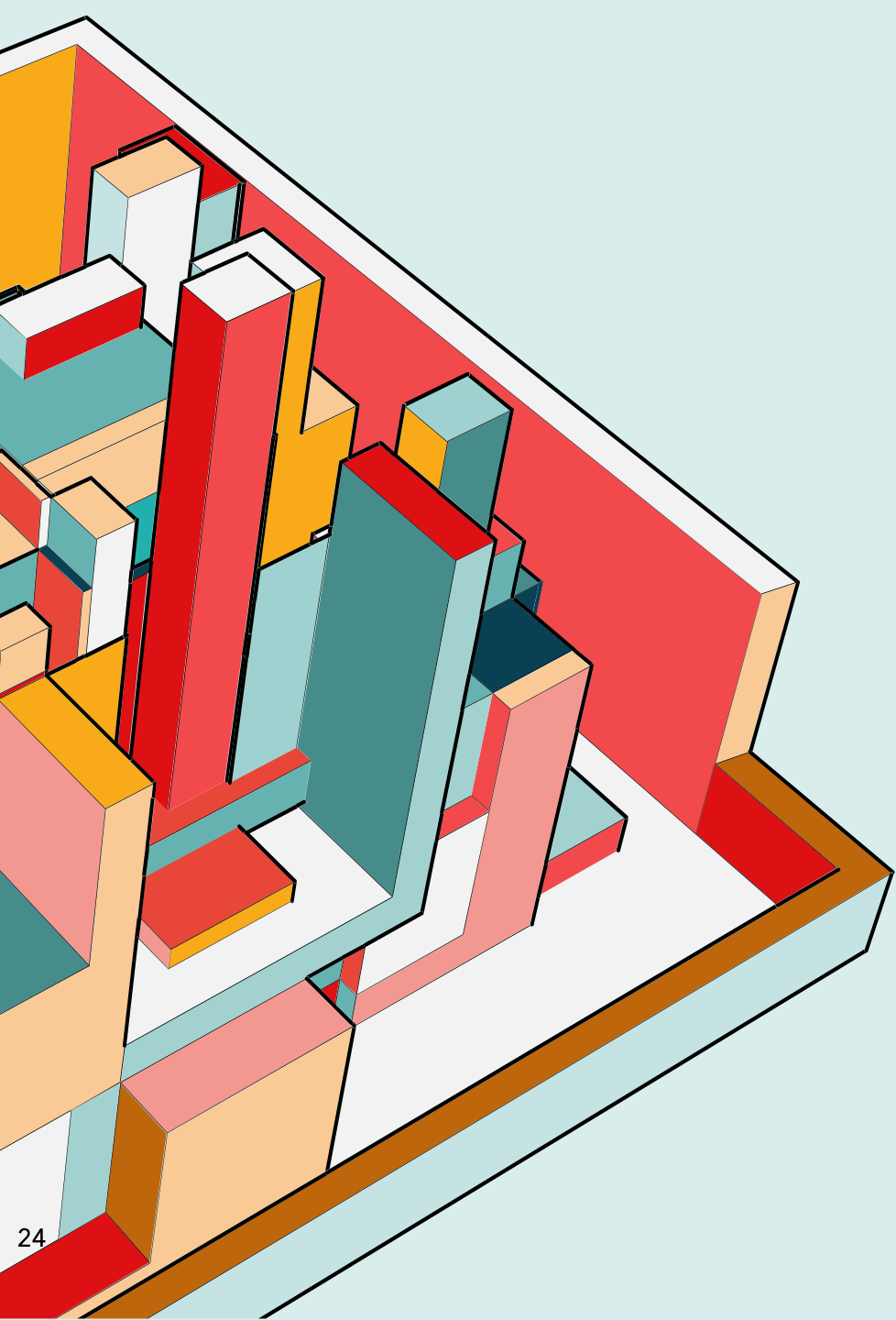
If there are any updates in round $|V|$, then there must be a negative cycle, and the nodes whose distances are updated must each be part of such a cycle.

If you want to know all of the nodes that are in a negative cycle, you can run $|V|$ extra rounds instead – every node that sees an update after round $|V| - 1$ is part of a negative cycle.

STILL TO COME IN PATHING

Both Dijkstra and Bellman-Ford are single-source shortest path algorithms – as mentioned last class, if you need to know many different shortest paths with various endpoints through the graph, we can do better than running these algorithms repeatedly.

The key algorithm for all-pairs shortest path is the Floyd-Warshall algorithm. However, it is an example of dynamic programming, which we will not address until after exam 2.



HEAPS

PRIORITY QUEUES

Several recent algorithms (Prim, Dijkstra, Huffman codes) have relied on an abstract data structure called a priority queue.

A priority queue allows items to be stored with a priority value attached, and must support the following basic operations:

- Returning the highest priority item
- Removing the highest priority item
- Adding a new item with arbitrary priority
- We have also made use of an “update” operation, but the basic PQ doesn’t have this. We’ll discuss it more in a bit.

The most common structure for implementing a priority queue is called a *heap*.

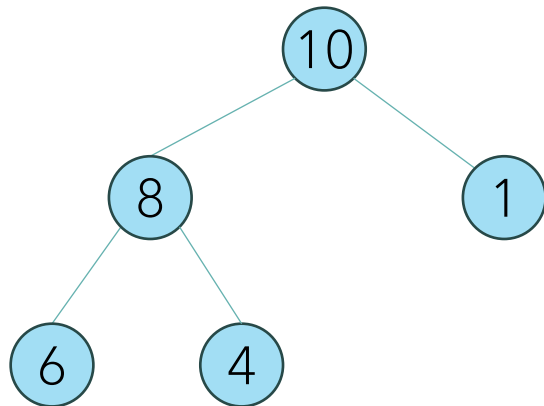
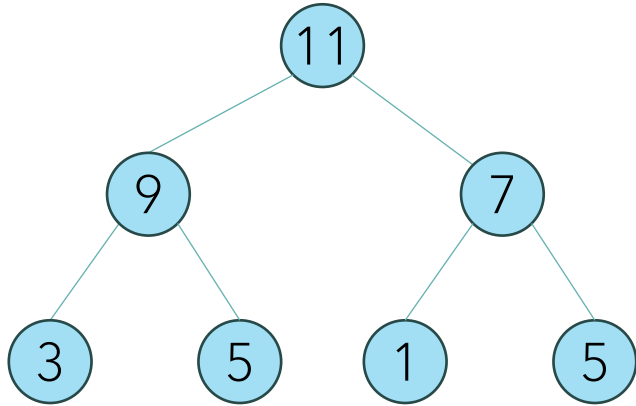
A HEAP IS A BINARY TREE

At its root 🧠 , a heap is just a binary tree upon which we impose a couple of simple restrictions:

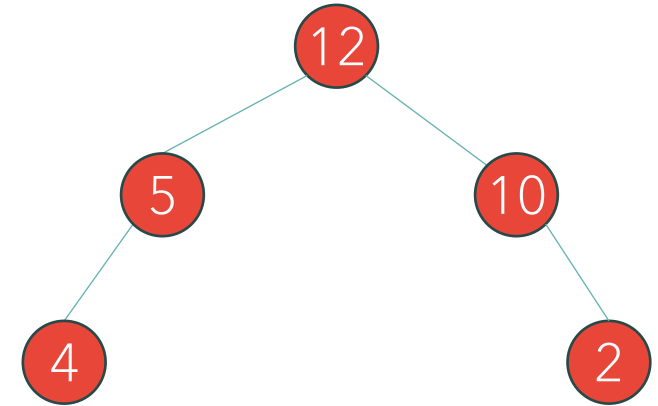
- The tree is *essentially complete* – that is, every level is full except the last one, and all of the leaves on that level are as far left as possible.
- Every node in the tree is greater than or equal to all of its descendants. (This is usually known as the *heap property*.)
 - Note a heap can also be constructed where ancestors are less than or equal to descendants. To avoid confusion, this is usually called a *min-heap*; Dijkstra and Prim both use a min-heap.

This is somewhat similar to a binary search tree, but a critical difference is that there is no relation between sibling nodes – either the left or right child may be larger.

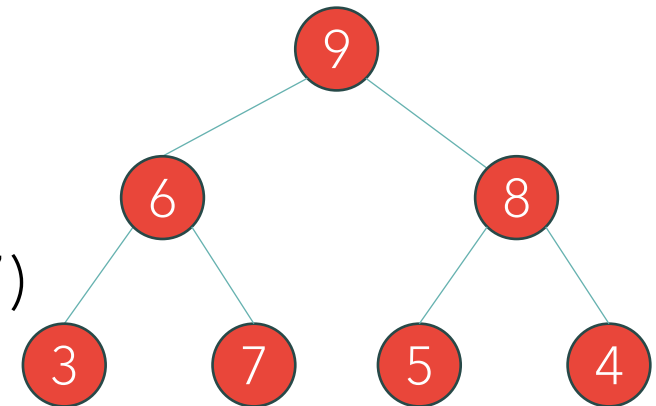
HEAP EXAMPLES



Not a heap -
violates shape
requirements



Not a heap -
violates heap
property ($6 < 7$)



IMPORTANT PROPERTIES OF HEAPS

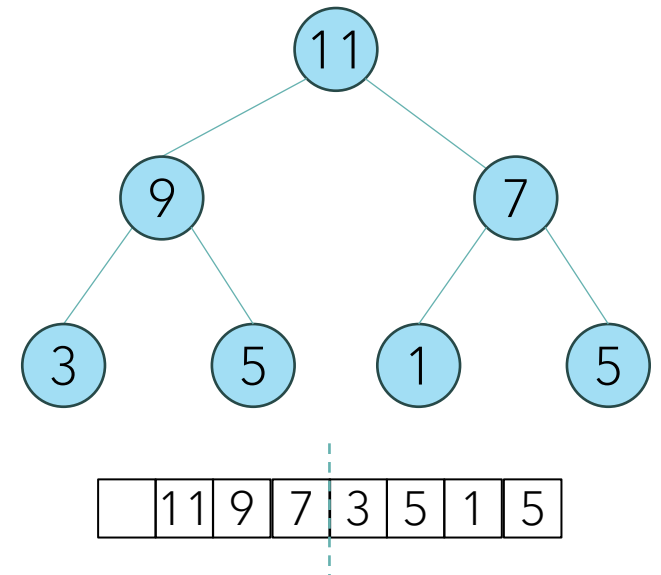
Every heap with n nodes has an identical structure, with height equal to $\lceil \log_2 n \rceil$.

The root of a heap is always its largest element.

Every induced subtree (i.e. a node and all of its descendants) of a heap is also itself a heap.

The most efficient way to implement a heap is in an array!

Typically we leave index 0 empty and put the root at index 1. Then the children of the node at index i are at indices $2i$ and $2i + 1$. All of the leaves are in the second half of the array.



WHY USE A HEAP FOR A PRIORITY QUEUE?

Generally, the action we need to take most often with a PQ is querying the highest-priority item.

In a heap, this is always the root! Thus, this is an $O(1)$ operation.

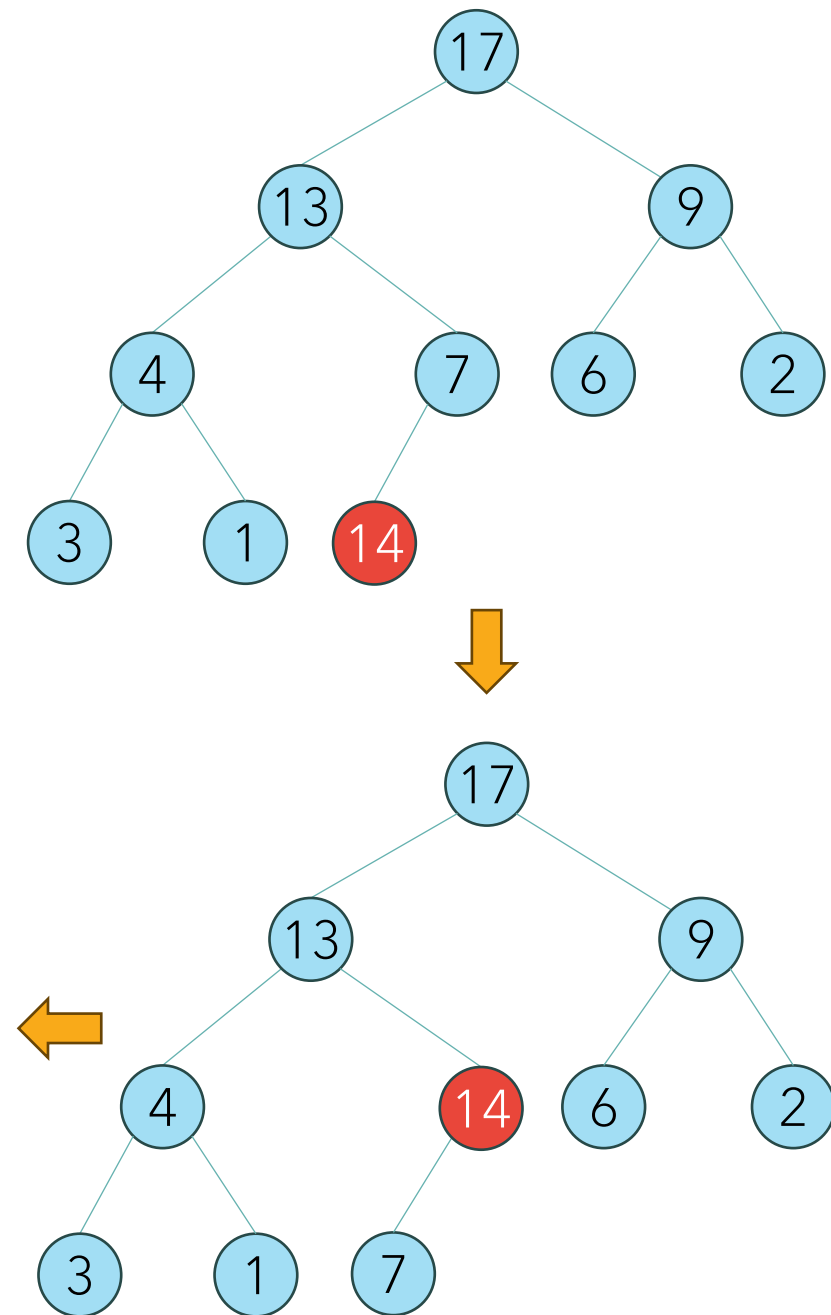
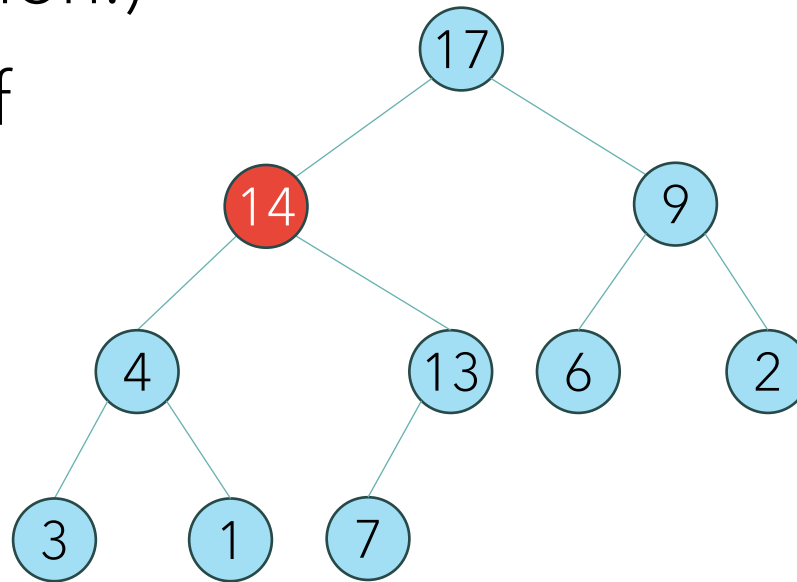
Note the improvement over a binary search tree, where this operation requires $O(\log n)$ time.

What about the other two actions?

INSERTING INTO A HEAP (“PUSH”)

The insertion process is fairly simple – we just add the new element in the next spot in the tree and then have it “swim” up to its appropriate spot. (This is often called a “heapify up” operation.)

The total number of comparisons and possible swaps is dependent on the tree height, and thus is $O(\log n)$.

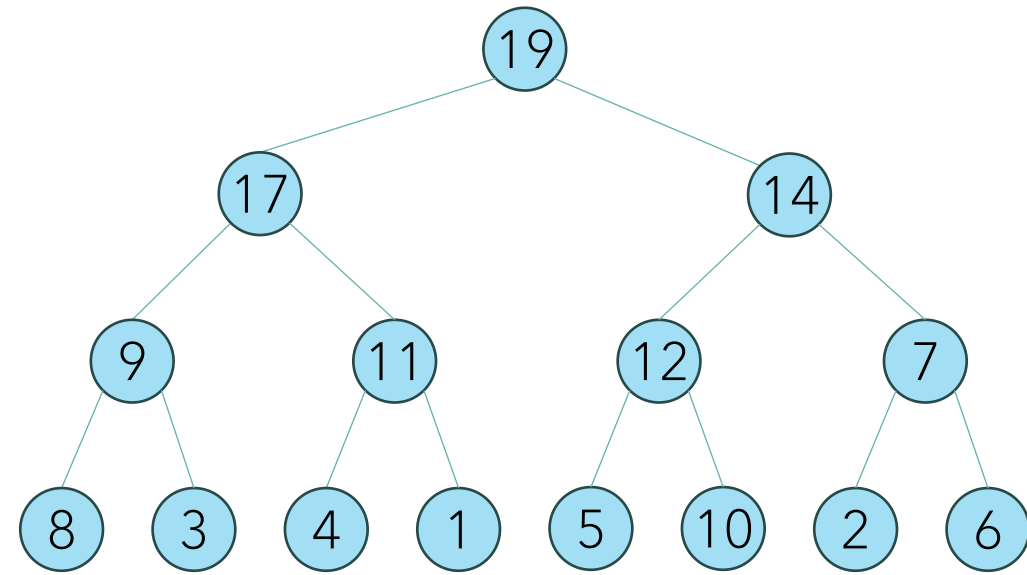


DELETING THE ROOT (“POP”)

Removing the highest-priority node is equally simple – we swap it with the last node, and then “heapify down” the new root until it satisfies the heap property.

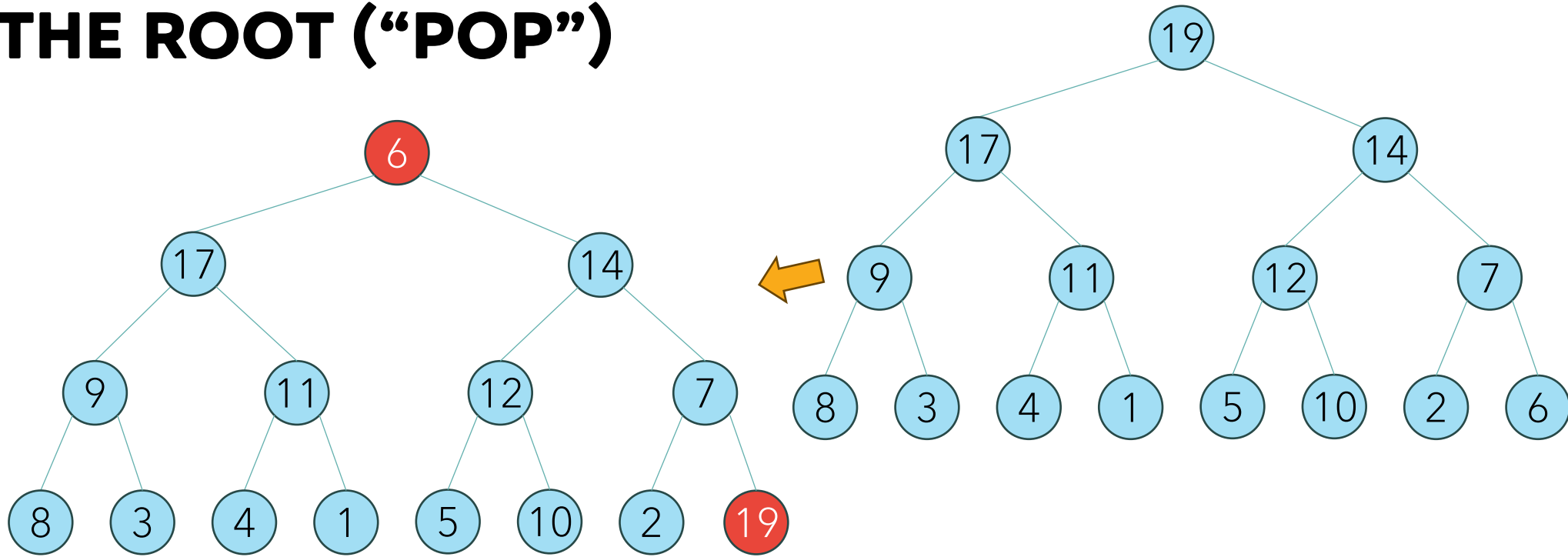
In this example, 6 (briefly) becomes the new root. Which value will it eventually take the place of in the heap? (Note that value will become its immediate parent.)

Answer now in the Submittity poll.



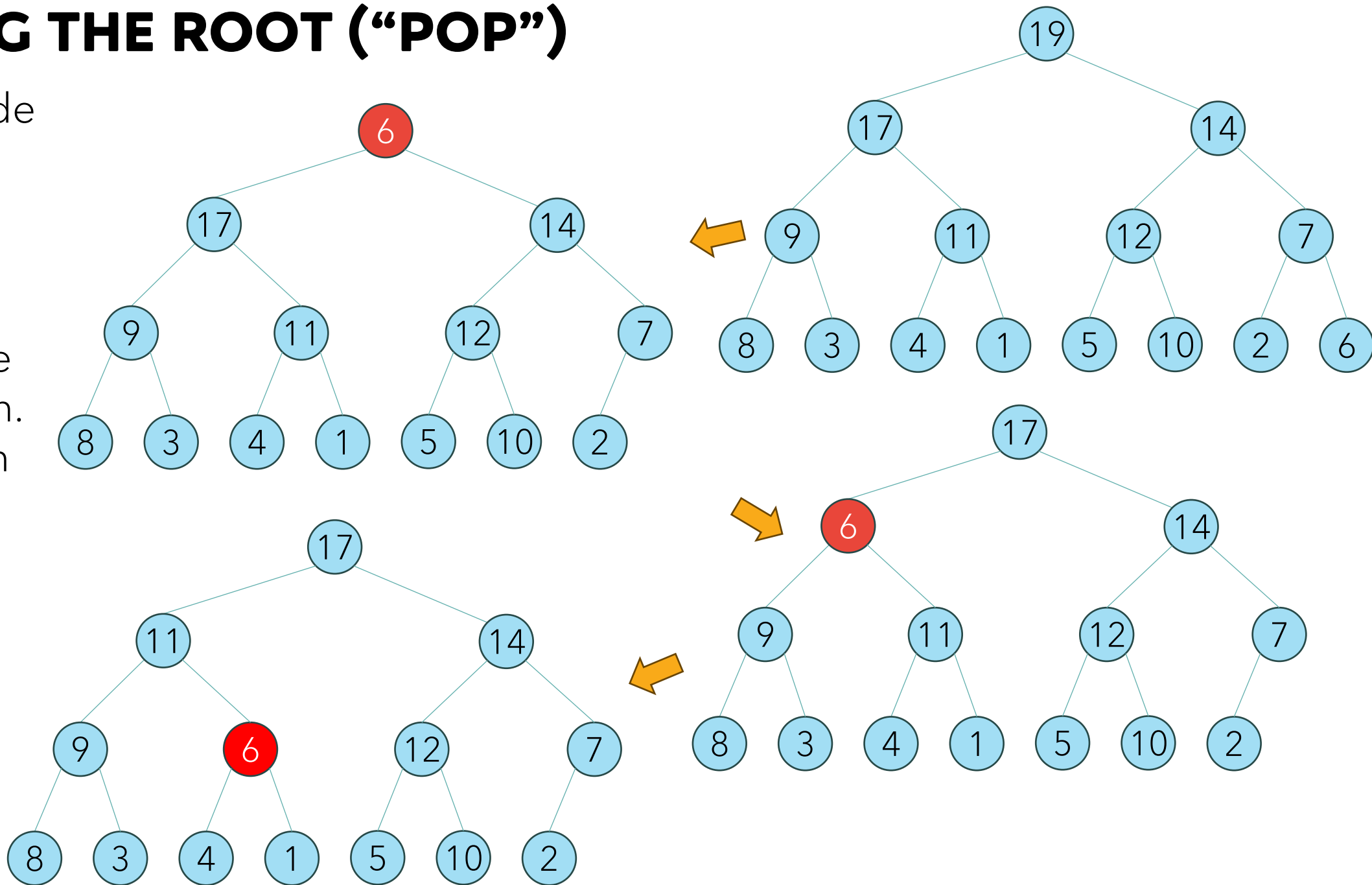
DELETING THE ROOT (“POP”)

When a node “sinks”, it must swap positions with the larger of the two children. This is again $O(\log n)$.



DELETING THE ROOT (“POP”)

When a node “sinks”, it must swap positions with the larger of the two children. This is again $O(\log n)$.



INITIAL CONSTRUCTION OF A HEAP

Given an arbitrary list of n values, how should we produce a heap from them in the first place?

One obvious approach is to start with an empty heap and just apply the insertion operation n times, referred to as “top-down” heap construction. The total time will be $\lg 1 + \lg 2 + \cdots + \lg n = O(n \log n)$.

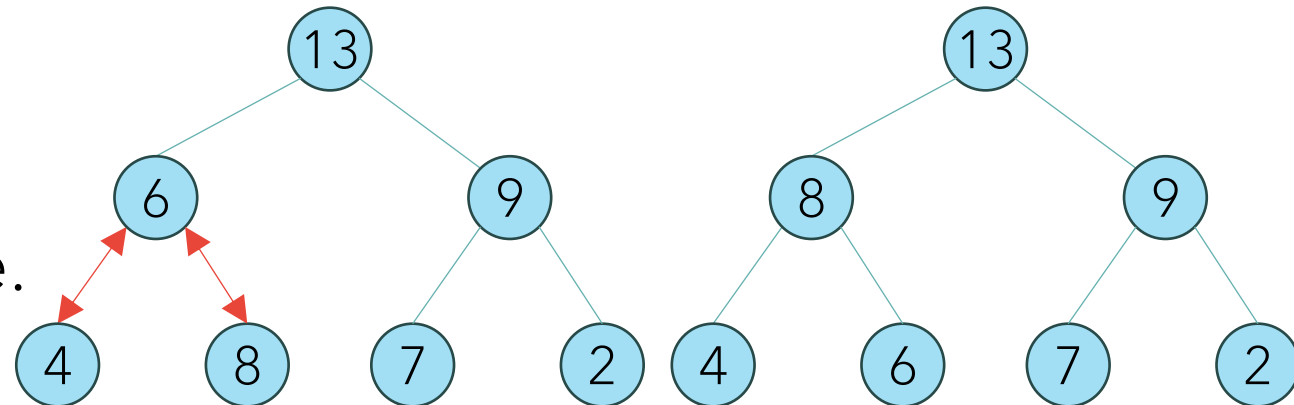
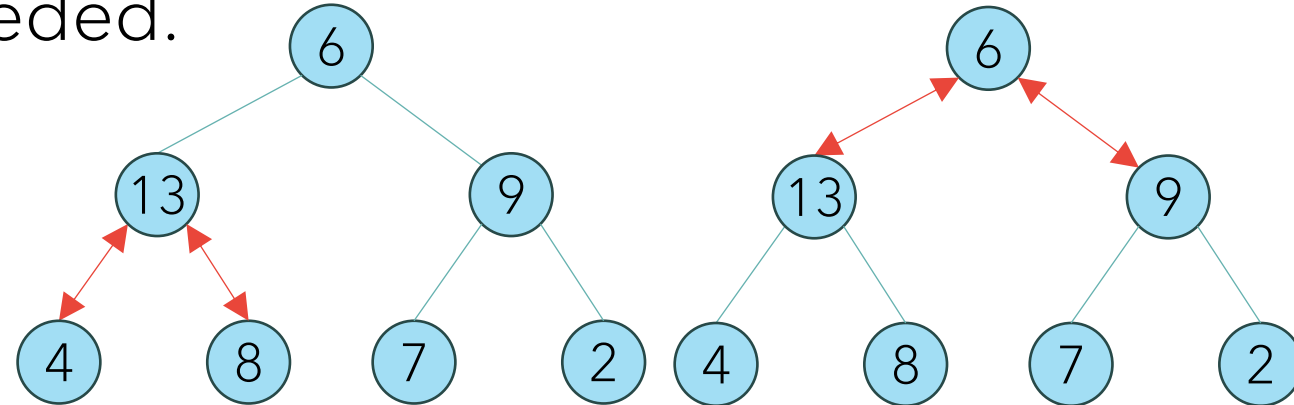
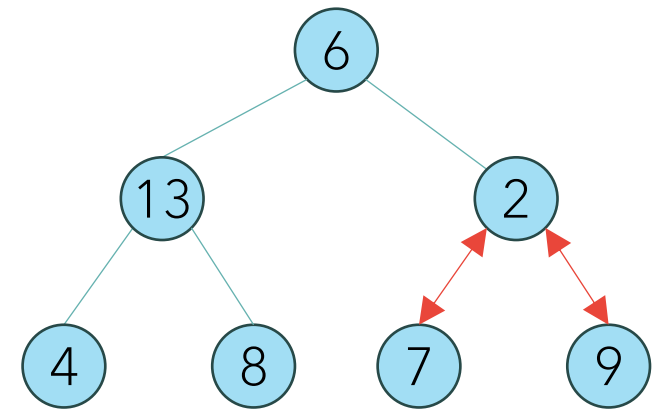
It turns out we can do better by using a “bottom-up” approach. It begins by just putting all of the values into the tree in the order received.

INITIAL CONSTRUCTION OF A HEAP

Once we have all of the values into a tree, we inspect each non-leaf node in reverse order, and heapify down as needed.

At level $h - 1$ we perform 2 comparisons per node, at level $h - 2$ it's at most 4, and each higher level adds 2 more to the worst case.

In general, at level i we need $2(h - i)$ comparisons per node.



TOTAL RUNTIME OF BOTTOM-UP METHOD

In the worst case:

$$T(n) = \sum_{i=0}^{h-1} \sum_{\text{level } i \text{ nodes}} 2(h-i) = \sum_{i=0}^{h-1} 2(h-i)2^i$$

since there are 2^i nodes at level i .

We can use $h = \lceil \log_2 n \rceil$ and the known solution for $\sum i \cdot 2^i$ to get a closed form for the above sum: $2(n - \log_2(n+1))$.

That is, bottom-up heap construction is $\Theta(n)$.

HEAPSORT

Once you have the concept and operations of a heap defined, a sorting algorithm immediately suggests itself:

1. Construct a heap with your data.
2. Invoke root deletion $n - 1$ times.

Construction is a linear time operation; root deletion requires logarithmic time. Thus heapsort runs in $\Theta(n \log n)$ time.

In practice, implementation of heapsort tends to be a bit faster than mergesort (and it's in-place!), but a bit slower than quicksort. Thus, it tends to get used only when the heap is valuable for other reasons.

HEAPSORT IN AN ARRAY

We'll use the same values as earlier: 6,13,2,4,8,7,9

Heap construction

Root extraction/deletion

6 13 2 4 8 7 9	13 8 9 4 6 7 2	2 6 7 4 8 9 13
6 13 9 4 8 7 2	2 8 9 4 6 7 13	7 6 2 4 8 9 13
6 13 9 4 8 7 2	9 8 2 4 6 7 13	4 6 2 7 8 9 13
13 6 9 4 8 7 2	9 8 7 4 6 2 13	6 4 2 7 8 9 13
13 8 9 4 6 7 2	2 8 7 4 6 9 13	2 4 6 7 8 9 13
	8 2 7 4 6 9 13	4 2 6 7 8 9 13
	8 6 7 4 2 9 13	2 4 6 7 8 9 13

BEFORE YOU LEAVE

